

Installing Source Code Analysis Tool

Pylint:

1. In VSC in einem Terminal „pip install pylint“ eingeben
2. Zum Ausführen „python -m pylint gilded_rose.py“ eingeben:

```
PS C:\Users\phili\Desktop\StaticCodeAnalysis\python> python -m pylint gilded_rose.py
***** Module gilded_rose
gilded_rose.py:49:0: C0303: Trailing whitespace (trailing-whitespace)
gilded_rose.py:1:0: C0114: Missing module docstring (missing-module-docstring)
gilded_rose.py:8:0: C0115: Missing class docstring (missing-class-docstring)
gilded_rose.py:8:0: R0205: Class 'GildedRose' inherits from object, can be safely removed from bases in python3 (useless-object-inheritance)
gilded_rose.py:13:4: C0116: Missing function or method docstring (missing-function-docstring)
gilded_rose.py:15:15: R1714: Consider merging these comparisons with 'in' by using 'item.name not in (AGED_BRIE, BACKSTAGE)'. Use a set instead if elements are hashable. (consider-using-in)
gilded_rose.py:14:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
gilded_rose.py:14:8: R1702: Too many nested blocks (6/5) (too-many-nested-blocks)
gilded_rose.py:14:8: R1702: Too many nested blocks (7/5) (too-many-nested-blocks)
gilded_rose.py:13:4: R0912: Too many branches (22/12) (too-many-branches)
gilded_rose.py:8:0: R0903: Too few public methods (1/2) (too-few-public-methods)
gilded_rose.py:50:0: C0115: Missing class docstring (missing-class-docstring)
gilded_rose.py:57:15: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
gilded_rose.py:50:0: R0903: Too few public methods (1/2) (too-few-public-methods)

-----
Your code has been rated at 6.82/10

PS C:\Users\phili\Desktop\StaticCodeAnalysis\python> 
```

Folgende Änderungen wurden durchgeführt:

1. class GildedRose(object) zu class GildedRose ändern
2. Vergleiche vereinfachen, statt !=, not in schreiben, also
if item.name != AGED_BRIE and item.name != BACKSTAGE
zu
if item.name not in (AGED_BRIE, BACKSTAGE)
3. F-String verwenden: statt return "%s, %s, %s" % (self.name, self.sell_in, self.quality), return f"{self.name}, {self.sell_in}, {self.quality}" schreiben
4. Adding docstrings:

```
class GildedRose:
    """Handles item updates"""
```

```
def update_quality(self):
    """Update quality and sell_in values"""
    for item in self.items:
```

```
class Item:
    """Represents an inventory item"""
```

5. Trailing whitespace: remove empty lines

6. + 7. Reduce nested blocks and branches:

```
def update_quality(self):
    """Update quality and sell_in values."""
    for item in self.items:

        if item.name not in (AGED_BRIE, BACKSTAGE):
            if item.quality > 0 and item.name != SULFURAS:
                if item.name == CONJURED:
                    item.quality -= 2
                else:
                    item.quality -= 1

        else:
            if item.quality < 50:
                item.quality += 1

                if item.name == BACKSTAGE:
                    if item.sell_in < 11 and item.quality < 50:
                        item.quality += 1

                    if item.sell_in < 6 and item.quality < 50:
                        item.quality += 1

            if item.name != SULFURAS:
                item.sell_in -= 1

            if item.sell_in < 0:
                if item.name == BACKSTAGE:
                    item.quality = 0
                elif item.name == AGED_BRIE:
                    if item.quality < 50:
                        item.quality += 1
                elif item.quality > 0 and item.name != SULFURAS:
                    if item.name == CONJURED:
                        item.quality -= 2
                    else:
                        item.quality -= 1
```